

APPUNTI DI STUDIO: ARCHITETTURA E ECOSISTEMA STM32

1. Introduzione ai Microcontrollori STM32

I dispositivi della famiglia **STM32** sono circuiti integrati a 32-bit prodotti da STMicroelectronics. A differenza di una semplice CPU, un microcontrollore STM32 integra al suo interno l'intero: memoria ram statica (SRAM) e flash (per lo storage del codice), interfaccia di debugging, uno o più cores, periferiche di IO integrate (Timer, ADC, DAC, I2C, SPI, UART...)

Il Portfolio STM32

La gamma si suddivide in diverse macro-categorie per rispondere alle differenti esigenze applicative:

1. **Mainstream:** Soluzioni bilanciate per usi generici (es. STM32F0, STM32G0).
2. **Ultra-low-power:** Progettati per ridurre al minimo i consumi energetici, ideali per dispositivi a batteria (es. STM32L0, STM32L4, STM32U5).
3. **High-performance:** Massima potenza di calcolo, frequenze elevate e accelerazione hardware (es. STM32F4, STM32F7, STM32H7, STM32N6 dotato di unità di elaborazione neurale/NPU).
4. **Wireless:** Soluzioni con connettività radio integrata

2. L'Ecosistema STM32Cube

L'**Ecosistema STM32Cube** è una suite completa che combina strumenti software e risorse hardware per guidare lo sviluppatore dall'idea iniziale fino alla prototipazione elettronica (Proof-of-Concept).

Struttura del Software e del Firmware (Synopsis)

La suite software si articola su più livelli (dal basso verso l'alto):

- **Drivers & HAL API (Hardware Abstraction Layer)**→API standardizzate e portabili.
- **Middleware**→Componenti software avanzati pre-integrati (es. stack di rete, file system, RTOS, librerie grafiche).
- **Function Pack**→ Esempi applicativi completi e pronti all'uso
- **Applicazioni utente**→ Sviluppate tramite PC, Mobile App o Cloud Services.

Esempio, scheda ST Nucleo

- **Alimentazione Flessibile:** Via USB o tramite una sorgente esterna.
- **Programmatore/Debugger Integrato:** Includono **ST-LINK**, la Nucleo è vista come memoria di massa
- **Fattori di Forma:** Disponibili in tagli da 32 pin (Nucleo-32), 64 pin (es. standard) o 144 pin (Nucleo-144, che includono opzioni Ethernet e USB OTG).
- **Plus:** Connettori **Arduino Uno v3** e **Morpho Headers** (per accedere a tutti i pin IO disponibili)

Schede di Espansione X-Nucleo

Le schede **X-Nucleo** si innestano sopra le schede Nucleo per aggiungere funzionalità periferiche specifiche, raggruppate in domini applicativi chiave:

- **Sense (Rilevamento):** Sensori di prossimità/distanza a tempo di volo (ToF) (es. VL53L5CX), sensori di movimento ed ambientali MEMS (accelerometri, giroscopi, magnetometri come la serie X-NUCLEO-IKS), e microfoni digitali.
- **Connect (Connettività):** Moduli Wi-Fi, Bluetooth Low Energy (BLE), Sub-1 GHz, GNSS (posizionamento satellitare) e tag/lettori NFC/RFID (es. basati su chip ST25).
- **Power Drive (Alimentazione e Movimento):** Driver per motori stepper, motori DC e motori brushless (BLDC), oltre a moduli di gestione energetica.
- **HMI (Interfaccia Uomo-Macchina):** Controllori touch e driver LED.

4. Strumenti Software: STM32Cube MX e STM32Cube IDE

STM32Cube IDE (Ambiente di Sviluppo)

È la piattaforma di sviluppo "all-in-one" basata su Eclipse che integra al suo interno le funzionalità di STM32Cube MX, un compilatore C/C++ e gli strumenti di compilazione e debug avanzato per i microcontrollori STM32.

STM32Cube MX (Configuratore Grafico)

È uno strumento visuale che consente di inizializzare l'hardware in modo guidato prima di scrivere il codice sorgente.

1. **Selezione del Target:** Permette di scegliere il componente partendo dal singolo chip (*MCU Selector*) o direttamente da una scheda di sviluppo ST (*Board Selector*).
2. **Vista Pinout & Configuration:** Mostra il chip grafico. L'utente seleziona le periferiche da un elenco e configura la loro modalità operativa. Il software assegna automaticamente i pin corrispondenti evitando conflitti, ma l'utente avanzato può mappare manualmente le funzioni bloccando i segnali sui pin desiderati (*Pin Locking*).
3. **Configurazione del Clock:** Consente l'impostazione visiva dell'albero dei clock, dei prescaler e dei PLL interni.
4. **Configurazione delle Interruzioni (NVIC):** Permette di attivare gli interrupt di sistema e delle periferiche, gestendo i livelli di preemption e sub-priority.
5. **Generazione del Codice:** Una volta completata la configurazione grafica, lo strumento genera automaticamente lo scheletro del progetto in linguaggio C, completo dei driver HAL necessari.

5. Esempi Pratici e Paradigmi di Sviluppo (Laboratorio)

Il materiale delinea i passaggi fondamentali per implementare alcune classiche applicazioni embedded:

A. "Hello World!" su Interfaccia Seriale (UART)

- **In STM32CubeMX:** Si seleziona la periferica **USART2** (solitamente connessa all'interfaccia ST-LINK sulle schede Nucleo) e si configurano i parametri di comunicazione (Baud rate, data bits, ecc.).
- **In STM32CubeIDE:** Per utilizzare la comoda funzione standard `printf()`, lo sviluppatore deve ridefinire (eseguire il *retargeting*) della funzione di basso livello `_write()` del compilatore C, reindirizzando i caratteri verso le API HAL della UART. Infine si apre un terminale seriale sul PC per osservare l'output.

B. Lampeggio LED (Led Blinking)

Soluzione 1: Polling con Delay (Sub-ottimale)

- Si configura il pin associato al LED integrato (es. **PA5** impostato come **GPIO_Output**).
- Nel ciclo principale (**main loop**), si inverte lo stato del pin sfruttando le funzioni HAL di toggling e si inserisce un ritardo bloccante (delay):

```
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
HAL_Delay(500); // Ritardo bloccante
```

Soluzione 2: Tramite Timer e Interruzioni (Efficiente)

- **Configurazione:** In STM32Cube MX si abilita un timer (es. **TIM10**), si impostano i parametri del contatore (Prescaler e Counter Period) per determinare il tempo di overflow, e si attiva l'interrupt globale del timer nel pannello NVIC.
- **Logica Software:** 1. Si dichiara una variabile globale con funzione di *flag*. 2. Si avvia il timer in modalità interrupt: **HAL_TIM_Base_Start_IT(&htim10);**. 3. Si effettua il *redefine* della funzione di callback legata al timeout del timer (**HAL_TIM_PeriodElapsedCallback**), all'interno della quale si imposta il flag a **true**. 4. Nel **main loop**, si controlla la presenza del flag: se attivo, si esegue l'azione (es. toggle del LED), si resetta il flag e si lascia la CPU libera di eseguire altre computazioni senza blocchi.

Soluzione 3: Tramite Modulazione a Larghezza di Impulso (PWM)

- Permette di variare la luminosità del LED anziché effettuare un semplice on/off.
- In CubeMX, si imposta il pin del LED (es. **PA5**) come canale di un timer compatibile (es. **TIM2_CH1**), si attiva la sorgente di clock e si imposta il canale in modalità **PWM Generation Ch1**. Si regolano il periodo del contatore e il valore di *Pulse* (Duty Cycle).
- Nel codice C, si avvia la generazione PWM tramite l'apposita API del timer.

C. Gestione di un Pulsante tramite Interrupt (Button Interrupt)

- **Configurazione:** Si verifica che il pin del pulsante utente (es. **PC13** sulla scheda Nucleo) sia configurato come **GPIO_EXTI13** (External Interrupt line). Nel modulo NVIC si abilitano le interruzioni sulle linee EXTI associate (linee **[15:10]**).
- **Flusso di esecuzione:** Si dichiara un flag globale; si ridefinisce la funzione di callback associata alla pressione del pulsante (**HAL_GPIO_EXTI_Callback**) per impostare il flag; infine, si gestisce l'evento in modo asincrono all'interno del loop principale scaricando la CPU dall'onere del controllo continuo (polling).

1. L'Evoluzione dell'Intelligenza Artificiale (AI)

- **Artificial Intelligence (AI, anni 50-80):** Tecnica che consenta a un computer di imitare il comportamento umano.
- **Machine Learning (ML, anni 90-2000):** Algoritmi e metodologie che migliorano nel tempo imparando direttamente dai dati forniti, superando l'approccio rigido delle formule matematiche tradizionali. Con l'arrivo del machine learning si passa da un approccio di **programmazione standard** (lo sviluppatore progetta un algoritmo specifico basandosi su regole predefinite) ad una **programmazione** più **empirica** (addestrare un modello con in grado di estrarre autonomamente pattern per risolvere il problema).
- **Deep Learning (DL, 2010-oggi):** Algoritmi di apprendimento capaci di estrarre significato da enormi quantità di dati. Utilizzano una gerarchia di livelli multipli che imitano il funzionamento delle reti neurali del cervello umano.

2. Architettura e Addestramento delle Reti Neurali (ANN)

Le Reti Neurali Artificiali si ispirano al sistema nervoso umano. Nel corpo umano, un neurone è composto da *dendriti* (ricezione di impulsi elettrici), *corpo cellulare* (elaborazione delle inferenze e decisioni) e *terminali assonali* (trasmissione dell'output).

Nella progettazione di un neurone artificiale si realizzano strutture simili usando una **funzione di attivazione** (la sigmoide è la classica, ma si usano anche la tangente iperbolica e la Rectified Linear Unit, che è molto efficiente), una rete neurale standard si compone di tre macro-livelli:

1. **Input Layer:** Riceve i dati d'ingresso.
2. **Hidden Layers (Livelli Nascosti):** Più stadi di trasformazione non lineare delle caratteristiche (caratteristiche di basso, medio e alto livello).
3. **Output Layer:** Fornisce l'approssimazione del risultato atteso.

Tipologie di Reti Principali

- **Multi Layer Perceptrons (MLP):** Reti dense standard.
- **Convolutional Neural Networks (CNN):** Sfruttano convoluzioni (2D o 3D), filtri e mappature per il riconoscimento di immagini e pattern complessi.
- **Recurrent Neural Networks (RNN):** A differenza delle CNN che non mantengono uno stato interno, le RNN integrano dei cicli (*loops*) che permettono alle informazioni di persistere nel tempo, rendendole ideali per dati sequenziali o serie temporali.

Il Processo di Apprendimento (Supervisionato)

1. **Forward Propagation (Inference Run):** I campioni del dataset (precedentemente diviso in *Training*, *Validation* e *Testing set*) vengono somministrati alla rete. Inizialmente i pesi sono casuali o predefiniti. L'output predetto viene confrontato con il valore reale (*ground truth*).
2. **Loss Function (Funzione di Costo):** Calcola l'errore o lo scostamento tra il valore predetto e quello reale. L'obiettivo dell'addestramento è minimizzare questo errore.
3. **Backpropagation & Gradient Descent:** Si torna indietro lungo la rete per aggiornare lentamente i pesi al fine di ridurre l'errore del ciclo successivo. Per evitare che il sistema non converga o impieghi troppo tempo, si utilizza una velocità di apprendimento adattiva (*Adaptive Learning Rate*). Un passaggio completo sull'intero set di addestramento definisce un'**epoca** (*epoch*).

Incertezza e Rilevamento Out-of-Distribution (OOD) nelle Reti Neurali

1. Il Limite dei Classificatori a Insieme Chiuso (Closed-Set)

- **Definizione:** I classificatori standard vengono addestrati su dati definiti "In-Distribution" (ID) e possono predire esclusivamente all'interno di un insieme fisso di classi note.
- **Il problema nel mondo reale:** Al momento del deployment, il modello può ricevere input che non appartengono alla distribuzione di addestramento (input **Out-Of-Distribution o OOD**), come difetti sconosciuti o sensori degradati.
- **Forzatura della predizione:** Un classificatore classico assegna *sempre* l'input OOD a una delle classi ID conosciute, inducendo in errore il sistema.
- **Softmax e Confidenza:** L'output della funzione Softmax non equivale a una confidenza calibrata. Anche se la probabilità massima (*Maximum Softmax Probability - MSP*) viene usata

come baseline , essa indica solo "quale classe nota ha vinto", portando spesso a una forte **sovra-confidenza** (*overconfidence*) anche su dati palesemente errati o rumorosi.

L'incertezza si divide principalmente in due categorie scientifiche:

A. Incertezza Aleatoria (Incertezza dei Dati)

- **Causa:** Deriva dalla casualità intrinseca nel processo di generazione dei dati (es. rumore termico dei sensori, variazioni fisiche come la turbolenza).
- **Riducibilità: Non** può essere ridotta raccogliendo più dati dello stesso tipo. Può essere attenuata solo migliorando l'hardware o i filtri di pre-processing.

B. Incertezza Epistemica (Incertezza del Modello)

- **Causa:** Deriva dalla mancanza di conoscenza del modello a causa di scarsità di dati in determinate regioni dello spazio logico, modelli deboli o classi mai viste.
- **Riducibilità: Sì**, può essere ridotta raccogliendo più dati di addestramento o migliorando l'architettura della rete.
- **Rilevanza per l'OOD:** È il tipo di incertezza direttamente collegato al rilevamento OOD. Quando il modello riceve un input OOD, l'incertezza epistemica è massima.

Le metodologie si dividono in due macro-approcci:

1. Metodi Integrati (Integrated Methods)

Modificano l'architettura o il processo di training della rete per renderla nativamente consapevole dell'incertezza (es. *Bayesian Neural Networks*, *Deep Ensembles*).

- **Svantaggi per l'Edge-AI:** Richiedono passaggi multipli di inferenza o l'archiviazione di più modelli in memoria, rendendoli scarsamente applicabili su dispositivi embedded a risorse limitate.

2. Metodi Post-Hoc

Vengono applicati a un modello pre-addestrato senza doverlo riaddestrare, manipolando i logit o le feature estratte. Sono i più adatti per i microcontrollori.

- **Metodi basati sui Logit (Energia Libera):** Calcolano l'energia libera del campione
- **Metodi basati sullo Spazio delle Feature (Distanza):** Misurano la distanza geometrica (es. Distanza di Mahalanobis o k-NN) tra le feature dell'input di test e i centroidi delle classi note nel livello di embedding.
- **Metodi Basati sulla Ricostruzione (Autoencoder):** Si addestra un Autoencoder *solo* su dati normali. Se l'errore di ricostruzione (es. MSE) tra l'input e l'output decodificato supera una determinata soglia, il dato viene flaggato come anomalia/OOD.
- **Outlier Exposure (OE):** Consiste nell'addestrare esplicitamente la rete a rifiutare i dati OOD introducendo "outlier sintetici o virtuali" nello spazio latente durante il training.

3. La Rivoluzione dell'Edge AI e del TinyML

I sistemi tradizionali si affidano al Cloud Computing per l'elaborazione dei dati. Tuttavia, l'enorme incremento di dispositivi IoT e di sensori sul campo genera una quantità massiva di segnali da elaborare. Questo scenario guida la transizione verso l'**Edge AI** e il **TinyML** (l'esecuzione dell'inferenza direttamente sul microcontrollore o sul sensore posizionato "all'estremità" della rete).

Benefici dell'Edge AI

- **Efficienza energetica e minor costo**
- **Latenza ultra-bassa:** Elaborazione in tempo reale fondamentale per applicazioni critiche.
- **Privacy e Sicurezza elevate:** I dati sensibili rimangono sul dispositivo senza essere condivisi
- **Ambiti applicativi principali:** Manutenzione predittiva industriale, monitoraggio dello stato di macchinari, Smart Home (rilevamento eventi), smart cities e automazione dell'IoT.

La difficoltà principale è data dalle **Risorse limitate**: Bisogna lavorare con pochissima memoria RAM (vincolata ai requisiti delle attivazioni dei livelli della rete) e Flash (vincolata al peso dei parametri e delle connessioni).

Architettura di un Nodo IoT per Edge-AI

Un tipico nodo IoT è composto da tre macro-livelli:

1. **Sensing:** Sensori locali (ambientali, inerziali, acustici, ottici).
2. **Computing:** Microcontrollori (MCU) a basso o ultra-basso consumo.
3. **Connectivity:** Sistemi di trasmissione a corto o lungo raggio (Bluetooth, Wi-Fi, ecc.).

La complessità dell'applicazione Edge-AI dipende direttamente dal tipo di sensore utilizzato e dalla frequenza dei dati generati (*Output Data Rate*):

- **Basso impatto:** Sensori ambientali (temperatura, umidità, pressione).
- **Medio impatto:** Sensori inerziali (accelerometri, giroscopi) e acustici (microfoni MEMS).
- **Alto impatto:** Sensori d'immagine/video e sensori 3D (ToF, Radar, Lidar).

4. La Suite Software STM32 per l'Edge AI

Per rimuovere le differenze tra i framework di sviluppo e nascondere le difformità hardware dei chip, STMicroelectronics fornisce la suite integrata **ST Edge AI Suite**.

- **STM32 AI Model Zoo**→modelli pre-addestrati e ottimizzati per dispositivi STM32
- **NanoEdge AI Studio**→utilizza AutoML, nasce per lavorare sulle time-series e per l'on-device learning
- **ST Edge AI Core (CLI)**→Per importare modelli dai principali framework ML, quantizzarli e compilarli in codice C ottimizzato per microcontrollori
- **ST Edge AI Developer Cloud**→per ottimizzare e validare modelli direttamente in cloud, effettuando benchmark in remoto su delle board farm
- **STM32Cube.AI Studio / X-CUBE-AI**→Graph Optimizer (completamente loss-less), Quantized Model Support (da FP32 a Int8), Memory Optimizer

Il Robot Didattico/Sperimentale "Will-E"

Il robot **Will-E** è stato concepito come piattaforma base aperta e a basso costo per lo sviluppo di sistemi robotici mobili.

Architettura Hardware di Will-E

È caratterizzato da un hardware semplice, economico e facilmente replicabile:

- **Telaio** in plexiglass
- **Meccanica e Attuazione:** 2 motori DC *brushed* con encoders e una ruota passiva.
- **Alimentazione:** Batteria Li-Po a 3 celle combinata con un convertitore DC-DC step-down (*STEVAL-ISA108V1*) che riduce la tensione a 7.5 V per alimentare i motori
- **Unità di Elaborazione Principale:** Scheda **Nucleo STM32F401RE** (MCU ARM Cortex-M4 a 84 MHz, 512 KB Flash, 96 KB RAM).
- **Schede di Espansione (Shields):**
 - **X-NUCLEO-IHM15A1:** Driver per due motori DC a spazzole basato sul chip (tensione 7-45 V, corrente fino a 1.5 Arms per motore), contiene i ponti H, ci permette di pilotare i motori in PWM.
 - **X-NUCLEO-IKS01a3:** Modulo sensori MEMS inerziali e ambientali che include l'accelerometro + giroscopio 3D *LSM6DSO* e il magnetometro 3D *LIS2MDL*.
 - **Custom Expansion Board:** Una scheda personalizzata dotata di connettori *Morpho*, connettori per gli encoder delle ruote, slot per scheda microSD e modulo Bluetooth.

Requisiti e Architettura del Firmware (FW) di Will-E

Il firmware implementato sulla scheda Nucleo deve assolvere in modo concorrente a diverse funzioni critiche:

1. **Ricezione e Decodifica dei Comandi:** Riceve stringhe in formato testo che rappresentano i comandi (muoversi avanti-indietro, impostare la velocità, eseguire rotazioni in loco o percorrere curve con un raggio ben preciso)
2. **Odometria (Dead Reckoning):** Stima e aggiorna continuamente le coordinate spaziali del robot (x, y, θ) basandosi sul feedback dei sensori.
3. **Data Logging:** Salva tutti i dati cinematici e dei sensori su una scheda microSD.
4. **Streaming Dati:** Invia i dati di navigazione/odometria in tempo reale tramite modulo Bluetooth.

3. Fondamenti di Controllo Motori e Feedback Cinematico

Pilotaggio dei Motori in PWM

I microcontrollori non possono variare agevolmente la tensione DC lineare in uscita senza complessi stadi DAC analogici. Si utilizza quindi la **Modulazione a Larghezza di Impulso (PWM)**:

- Il segnale ha una forma d'onda rettangolare a frequenza fissa, in cui varia il **Duty Cycle** (rapporto tra il tempo di ON e il periodo totale).
- La velocità del motore varia in modo proporzionale al Duty Cycle impostato (es. 25%, 50%, 75%).

Case Study 1: Robot Umanoidi e il Progetto AMICO (Robot NAO)

- **Visione Artificiale ed Riconoscimento Facciale:** Utilizzando la telecamera commerciale *OpenMV* (basata su un microcontrollore ad alte prestazioni *STM32H7*), è stato integrato un firmware ST per il **riconoscimento facciale** e lo *Smile Detection* (rilevamento del sorriso) per guidare o far reagire il robot in base alle espressioni dell'utente.

- **Controllo Vocale via Bluetooth (BTLE):** Sfruttando le schede *SensorTile* e *BlueCoin*, è stato implementato il controllo vocale. La scheda *SensorTile* monta una rete neurale ricorrente (**RNN**) addestrata localmente per riconoscere 6 comandi vocali, trasmettendoli via Bluetooth Low Energy a NAO.

Case Study 2: Re-engineering del Robot Umanoide KONDO KHR-3HV

Il robot KONDO KHR-3HV è un kit umanoide dotato originariamente di 17 servomotori attivi (espandibili a 22) che comunicano tramite il protocollo seriale e PWM proprietario **ICS 3.5** (Half-Duplex su singola linea dati, livelli logici a 5V TTL, velocità fino a 1.25 Mbps).

Il team di sviluppo (in collaborazione con l'Università di Catania) ha effettuato il porting completo dell'elettronica sostituendo la scheda madre nativa (Renesas) con una **ST Nucleo STM32F446RE**. Il processo ha richiesto:

1. **Sniffing e Reverse Engineering del Segnale:** Tramite oscilloscopio e analizzatori seriali, sono state decodificate le stringhe esadecimali dei movimenti (composte da array contenenti preambolo, numero di frame, coppie di byte invertite per la posizione del servo e checksum).
2. **Adattamento Hardware:** È stata progettata una scheda di espansione personalizzata per la Nucleo per gestire i livelli di tensione (portando i segnali UART da 3.3V a 5V TTL richiesti dal protocollo ICS 3.5), i collegamenti daisy-chain dei servo, la telemetria della batteria e l'interfaccia con la scheda di supporto *BlueCoin*.

Esempi di AI nei Sensori

Una delle frontiere più importanti studiate è l'uso delle **ISPU (Intelligent Sensor Processing Units)**, come il chip *LSM6DSO16IS* (accelerometro + giroscopio dotati di 32KB di RAM programmabile interna). Le ISPU integrano la capacità di calcolo direttamente nel sensore, elaborando algoritmi AI a bassissimo consumo energetico senza svegliare la CPU principale del robot.

Come esempio vedi l'implementazione di un modello per la Fall Detection nell'ISPU (lavorando con pitch e roll ottenuti dalla fusione dei dati di giroscopio e accelerometro) che raggiunge un'accuratezza del 94% e che ha dimensione finale pari a ~28Kb.

Il Microcontrollore di Nuova Generazione STM32N6

Il trend evolutivo vede la sostituzione di Single Board Computer energivori (come Raspberry Pi 4) con microcontrollori industriali dedicati come l'**STM32N6** (core *ARM Cortex-M55* affiancato dall'acceleratore hardware proprietario **ST Neural ART NPU** da 600 GOPS). Questo processore abilita funzionalità avanzate direttamente a bordo:

- Integrazione di telecamere a otturatore globale per tracciare oggetti in rapido movimento.
- Supporto nativo ad algoritmi di Computer Vision avanzati, Object Detection e SLAM.

1. INTRODUZIONE ALLA NAVIGAZIONE ROBOTICA

La navigazione dei robot può essere sintetizzata in due compiti o task principali:

- **Obstacle Avoidance (OA - Evitamento degli Ostacoli)** (*pensa ai robot che vanno a caso*)
- **Point to Point Navigation (P2P - Navigazione da Punto a Punto)** (*serve fare la mappa*)

I sistemi di navigazione P2P si dividono in due grandi categorie:

1. **Global Navigation System (Sistemi di Navigazione Globale):** Conosco già la mappa e uso algoritmi già pronti
2. **Local Navigation System (Sistemi di Navigazione Locale):** Non conosco la mappa, mi devo orientare mentre mi muovo
 - Si usano gli algoritmi Bug:
 - Bug-0: Towards e Circumnavigate (se l'ostacolo ha una forma irregolare è più complicato)
 - Tangent Bug: muovo il robot minimizzando la distanza tangente locale richiede una funzione specifica di "follow boundary" (segui il confine) per superare l'oggetto.
 - Towards and Tangent: mi muovo verso l'obiettivo (Head towards goal) and se incontro un ostacolo, scelgo la direzione di deviazione che presenta l'angolo minore. Seguire tale direzione finché non è nuovamente possibile muoversi direttamente verso l'obiettivo (*leave point*).

Algoritmi SLAM: uso il LIDAR e le Macchina a Stati:

- posso evitare gli ostacoli avanzando fino ad un limite di sicurezza e poi ruotando a destra o sinistra
- Se gli ostacoli sono più bassi del raggio d'azione del LIDAR, il robot rileva il problema attraverso una macchina a stati dedicata ed esegue una manovra di retromarcia (*GO_BACK*) o correzione d'errore (*ROTATE_ERROR*).

Il rumore dei sensori introduce inevitabilmente errori nella stima della posa e della mappa. Per risolvere questo problema, l'architettura SLAM necessita di:

1. **Loop Detection / Place Recognition:** La capacità del sistema di riconoscere se sta transitando in un luogo già visitato in precedenza.
2. **Loop Closure (Chiusura del Loop):** Il processo di correzione dell'intera mappa e della traiettoria accumulata, sfruttando l'informazione che i punti di riferimento (*landmarks*) sono stazionari. La correzione della posa della telecamera/sensore implica automaticamente la correzione della mappa.

Tecnologie correlate ("Siblings")

- **SfM (Structure from Motion):** Ricostruisce una struttura 3D da un insieme di immagini 2D. Non richiede odometria, non ha vincoli computazionali stringenti e non lavora in tempo reale.
- **VO (Visual Odometry):** Stima l'odometria calcolando i cambiamenti di posa della telecamera tra fotogrammi consecutivi. È efficiente ma soffre della mancanza di *Loop Closure* e di una mappatura globale.

L'Evoluzione con l'Optical Flow (Versione STMicroelectronics)

La ricerca industriale (ST) ha sviluppato schede hardware dedicate (come la scheda *STEREO-CAM* dotata di sensori d'immagine *FoX NIR* con acceleratore hardware di **Optical Flow**) per potenziare l'ORB-SLAM:

- **Version 1 (Standard):** Il tracking si basa sulla classica estrazione di descrittori ORB, sul matching dei punti chiave e sulla decisione dei *Key Frame*.

- **Version 2 (OF-based):** Sfrutta l'accelerazione hardware della scheda: i vettori di movimento (*Motion Vectors*) calcolati direttamente dall'Optical Flow dei sensori FoX vengono convertiti e usati direttamente come *KeyPoints*. Questo riduce drasticamente l'overhead computazionale legato all'estrazione continua dei descrittori software tradizionali.
- **Sviluppi in ORB-SLAM3:** Introduzione della fusione sensoriale profonda mediante l'integrazione di dati IMU (accelerometro e giroscopio 3D) per raffinare la scala della mappa e gestire l'unione di mappe multiple (*Map Merging*).

BRAIN COMPUTER INTERFACES (BCI) E RICONOSCIMENTO DEI MOVIMENTI OCULARI

1.1 Introduzione e Sfide delle BCI

Una **Brain-Computer Interface (BCI)** è un sistema che consente la comunicazione diretta tra il cervello umano e dispositivi esterni.

- **Importanza:** Rappresenta una tecnologia rivoluzionaria per individui con gravi disabilità motorie che preservano esclusivamente i movimenti oculari.
- **Sfide principali dei sistemi BCI classici:**
 - Limitato numero di dimensioni di controllo.
 - Bassa accuratezza nella classificazione.
 - Necessità di eseguire i comandi in modo sincrono con uno stimolo esterno.
 - Lunghi periodi di addestramento richiesti all'utente per controllare il sistema.

1.2 Approcci di Acquisizione: Invasivi vs Non-Invasivi

L'attività cerebrale può essere rilevata tramite diverse metodologie, classificate in base al livello di invasività chirurgica:

1. **Sistemi Invasivi (es. Neuralink, Braingate, Synchron):**
 - *Pro:* Elevata accuratezza e risoluzione del segnale; funzionamento in tempo reale; adatti a compiti complessi (es. controllo di arti robotici).
 - *Contro:* Richiedono procedure chirurgiche; costi elevatissimi; barriere etiche e regolatorie.
2. **Sistemi Non-Invasivi (es. Emotiv, NeuroSky, Muse, OpenBCI):**
 - Sfruttano principalmente l'**Elettroencefalografia (EEG)** tramite elettrodi posizionati sullo scalpo. Rilevano le onde cerebrali categorizzate per frequenza
 - *Pro:* Sicuri, facili da usare ed economici per applicazioni base.
 - *Contro:* Risoluzione limitata del segnale; forte sensibilità a rumori e artefatti (es. movimenti muscolari); performance in tempo reale limitate nei sistemi commerciali standard.

1.3 Innovazione: Biosensori Elettrostatici e IA On-Board

Per superare i limiti dei sistemi commerciali (scarsa portabilità, assenza di elaborazione IA locale, latenza), la soluzione proposta combina **Biosensori STMicroelectronics** e **Intelligenza Artificiale Edge**.

- **Principio di funzionamento del Biosensore:** Si basa sulla rilevazione della carica elettrostatica. Quando due materiali vengono a contatto e si separano, avviene uno spostamento di elettroni (uno si carica positivamente, l'altro negativamente). Le fluttuazioni del campo elettrico circostante vengono intercettate dagli elettrodi e amplificate dal circuito integrato.

1.4 Pipeline di Elaborazione e Riconoscimento dei Movimenti Oculari

Il sistema è progettato per riconoscere 4 classi di movimenti: **Involuntary Blink (IB)**, **Voluntary Blink (VB)**, **Looking LEFT** e **Looking RIGHT**.

1. **Pre-processing (Filtraggio):** Il segnale grezzo passa attraverso una cascata di filtri per rimuovere il rumore di rete e la deriva di fondo:
 - Filtro Passa-Alto
 - Filtro Notch
 - Filtro Passa-Basso
2. **Event Detection (Rilevamento del picco):** Basato su un algoritmo a **z-score** che analizza le deviazioni del segnale e attiva la raccolta di una finestra temporale di 300 campioni (pari a un tempo di acquisizione di 1.25s con $f_s = 240 \text{ Hz}$).
3. **Classificazione (1D CNN):** Una rete neurale convoluzionale monodimensionale esegue l'inferenza direttamente sul microcontrollore.
 - *Impronta hardware su STM32H7:* Richiede solo **18.2 KB di Flash**, **6.6 KB di RAM** e una complessità computazionale di appena **21K MACC**. L'inferenza locale richiede solo 0.72 ms.

1.5 Personalizzazione del Modello (Personalization)

L'accuratezza del modello crolla vistosamente quando viene testato su un individuo "non visto" (unseen) durante il training generalizzato (es. l'accuratezza per "Persona 1" scende al 57.14%).

- **Metodologia:** Per risolvere questo problema, il modello viene diviso in una parte non-trainabile (addestrata sui dati di 8 individui) e una parte **trainabile** che viene calibrata tramite *fine-tuning* sui segnali dell'utente finale.
- **Risultati:** La personalizzazione porta l'accuratezza media a un incremento del **+9.97%**, permettendo a diversi utenti precedentemente problematici di raggiungere il **100% di accuratezza** nel riconoscimento dei comandi oculari.

INTERFACCE VOCALI (VUI) ED EDGE AI SU STM32

2.2 Il Keyword Spotting (KWS) a Risorse Limitate

Il **Keyword Spotting (KWS)** è l'algoritmo "sempre in ascolto" progettato per identificare una specifica parola chiave escludendo il parlato comune o i rumori di fondo ambientali.

- **Target Hardware di Riferimento:** Piattaforma indossabile **SensorTile** (dotata di microcontrollore **STM32L476 @80MHz**, 1MB di Flash, 128KB di RAM e un singolo microfono digitale). Il sistema non implementa pre-processing avanzati e pesanti come il beamforming o la localizzazione della sorgente, per preservare le risorse energetiche.
- **Estrazione delle Feature (MFCC):** L'audio a 16 kHz viene convertito in coefficienti **MFCC (Mel Frequency Cepstral Coefficients)**.

2.3 Gestione dei Dataset ed Etichettatura Debole (Weak Labelling)

L'accuratezza dei modelli KWS dipende fortemente dall'eterogeneità dei dati. Se un modello viene addestrato solo con accenti americani, fallirà con parlanti nativi francesi, italiani o asiatici a causa delle peculiarità fonetiche.

- **Dataset di Partenza:** Si utilizza il *Google Speech Commands Dataset* (65.000 enunciati da 1 secondo per 30 parole corte) integrato con registrazioni locali effettuate tramite SensorTile.
- **Data Augmentation:** Per incrementare la robustezza e prevenire l'overfitting, il dataset originale viene esteso massicciamente (es. da 21.154 parole a 172.326 parole) applicando variazioni di pitch, time stretching e mix di rumore ambientale.
- **Performance su STM32L4:** La classificazione binaria (Parola chiave "Marvin" vs "Other") raggiunge un'accuratezza del **96.5%**. Convertita tramite lo strumento **STM32Cube.AI**, la rete occupa appena **16.5 KB di Flash**, **2 KB di RAM** e richiede **15 ms** per l'inferenza.

2.4 Controllo Robotico Tramite Infezioni Consecutive

Per implementare il controllo vocale di un braccio robotico o di un piccolo rover, l'architettura a basso consumo richiede l'esecuzione di **due inferenze consecutive**:

1. **Prima Inferenza:** L'algoritmo monitora costantemente l'audio alla ricerca della wake-word ("Marvin").
 2. **Seconda Inferenza:** Solo se la wake-word viene rilevata, il sistema si attiva per classificare il comando effettivo di movimento all'interno di un set predefinito (*Up, Go, Left, Right, Stop*).
- Una implementazione basata su Recurrent Neural Network (RNN) in virgola mobile (FP32) per la gestione sequenziale di questa catena richiede **295 KB di Flash** e solo **0.52 KB di RAM**, rendendo superflua la compressione o la quantizzazione su microcontrollori di fascia media.

Interfacce Cervello-Computer (BCI) e Controllo Robotico Embedded

1. Architettura di Sistema del Progetto OpenBCI

L'obiettivo del progetto descritto è la realizzazione di un sistema embedded in grado di acquisire segnali biologici dalla testa e tradurli in comandi motori per una piattaforma robotica a 3 ruote chiamata "WILL-E".

Il sistema si compone di:

1. **Hardware di acquisizione (Helmet):** Casco stampabile in 3D in grado di ospitare fino a 16 canali per elettrodi.

2. **Scheda Custom STM32:** Basata sul microcontrollore **STM32L475** (84MHz, 92KB SRAM, 512KB FLASH) integrata con un Front-End Analogico (AFE), un convertitore Analogico-Digitale (ADC), memoria SD Card, Bluetooth e protezione ESD.
3. **Software:** Per l'elaborazione dei segnali in tempo reale e il data logging.

2. Segnali Elettro-Oculografici (EOG) e Raccolta Dati

Invece dei classici segnali EEG (elettroencefalografici), il sistema si focalizza sui segnali **EOG (Elettro-Oculografia)** sfruttando sensori posizionati nella regione fronto-polare (siti elettrodi Fp1 e Fp2).

- **Target Biologici:** Il sistema punta a riconoscere 4 specifici eventi oculari: battiti volontari (*Voluntary Blinks* - VB), battiti involontari (*Involuntary Blinks* - IB), strizzate dell'occhio sinistro (*Left Winks* - LW) e dell'occhio destro (*Right Winks* - RW).
- **Dataset:** Raccolto su 8 soggetti sani senza precedenti esperienze BCI; l'addestramento è stato guidato da stimoli visivi a schermo (tranne per i battiti involontari, registrati spontaneamente) ottenendo un dataset bilanciato di 2600 eventi etichettati.

3. La Pipeline del Firmware sul Microcontrollore

L'algoritmo completo gira interamente a bordo dell'MCU STM32 secondo i seguenti step sequenziali:

[Sensori EOG] -> [ADC] -> [Pre-processing] -> [Event Detection] -> [CNN Classification] -> [Comandi Robot]

A. Pre-processing (Filtraggio Digitale)

Ha lo scopo di eliminare la componente continua (DC offset) e i disturbi della frequenza di rete elettrica (50Hz/60Hz).

- **Filtro Passa-Banda:** Filtro IIR real-time con frequenze di taglio
- **Filtro Notch:** Filtro IIR elimina-banda tarato sulla frequenza di rumore a 50Hz.

B. Rilevamento dell'Evento (Event Detection)

- **Algoritmo:** Basato sulla metrica statistica dello **Z-score** per individuare i picchi negativi associati al movimento oculare.
- **Finestratura:** Il superamento della soglia di Z-score avvia la raccolta di una finestra di **200 campioni** temporali da inviare al classificatore.

C. Classificazione dell'Evento via Rete Neurale (CNN)

I 200 campioni estratti vengono processati da una rete neurale convoluzionale (CNN) ultra-leggera ottimizzata per l'edge computing.

- **Footprint di Memoria (STM32L475):**
 - **Flash occupata:** Solo 66 KB
 - **RAM occupata:** Solo 11 KB
- **Complessità computazionale:** 270K MACC (Moltiplicazioni e Accumuli).
- **Accuratezza del modello:** **99%** complessiva (valutata tramite matrice di confusione sui 4 stati biologici).

4. Controllo Robotico (WILL-E)

Mappatura dei Comandi Inviati al Robot:

- **VB (Battito Volontario):** Il robot avanza di 20 cm (*Go ahead*).
- **IB (Battito Involontario):** Nessuna azione (*Nothing to do*), filtrato come rumore biologico di fondo.
- **LW (Strizzata Sinistra):** Il robot ruota di 45° a sinistra (*Turn left*).
- **RW (Strizzata Destra):** Il robot ruota di 45° a destra (*Turn right*).

2. CASE STUDY 1: CLASSIFICAZIONE DELLE SCENE ACUSTICHE (ASC) SU STM32L4

La **Acoustic Scene Classification (ASC)** consiste nel classificare l'ambiente circostante partendo esclusivamente dai suoni che produce (es. identificare se ci si trova in un ufficio, su un autobus o a casa).

Dataset e Tassonomia

- **Origine:** Basato sulle sfide internazionali DCASE, integrato da una campagna di raccolta dati interna a STMicroelectronics (~30 ore totali di registrazioni PCM a 16KHz, mono, 16-bit).
- **Macro-classi (Tassonomia aggregata):** Per l'implementazione sul microcontrollore, le scene sono state raggruppate in 3 categorie principali:
 1. *Indoor* (casa, ufficio, ristorante, centro commerciale).
 2. *Outdoor* (centro città, parco, area residenziale).
 3. *In-vehicle* (autobus, auto, treno, metropolitana).

Pipeline del Firmware ed Estrazione delle Feature

Il segnale audio catturato dai microfoni digitali MEMS subisce una pipeline sequenziale in tempo reale sul microcontrollore:

1. **Finestratura Audio (Framing):** Finestre temporali di **1024 campioni** (pari a 64 ms a 16KHz) con un **overlap del 50%** (512 campioni / 32 ms) gestite in modo efficiente tramite buffer gestiti in DMA (*Direct Memory Access*).
2. **Estrazione delle Feature (LogMel Energies):** * Si applica una finestra asimmetrica di Hann (*Asymmetric Hann windowing*).
 - Si calcola la FFT veloce e la magnitudo complessa usando le librerie matematiche ARM CMSIS-DSP (**ARM rfft fast f32** e **ARM cmplx mag f32**).
 - Il profilo di potenza viene mappato su banchi di filtri Mel (*MEL Filter Banks*) e ne viene calcolato il logaritmo dell'ampiezza.
 - **Output:** Viene calcolata una colonna di **30 energie Log-Mel** ogni 32 ms. Accumulando le colonne su un arco temporale di 1024 ms, si ottiene una **Matrice di Feature (Spettrogramma) di dimensione 30x32**.
3. **Inferenza della Rete Neurale:** La matrice 30x32 alimenta una rete neurale convoluzionale (CNN).
4. **Post-processing (Time Filtering):** Un filtro passa-basso temporale (*LP time filter*) stabilizza le predizioni, incrementando la confidenza del sistema e attenuando i falsi positivi durante le transizioni tra ambienti.

Architettura del Modello CNN ed Ottimizzazione Edge

La rete neurale è una struttura leggera sequenziale:

- Livelli convoluzionali 2D, seguiti da Max Pooling, Batch Normalization e Dropout.
- Livelli Dense (completamente connessi) fino all'output Softmax a 3 classi.
- **Tool di conversione:** È stato impiegato **X-CUBE-AI** (il tool automatico di ST per mappare i modelli Keras/TensorFlow in codice C ottimizzato per STM32).
- **Quantizzazione (PTQ):** Il modello è stato convertito da precisione floating-point a 32-bit (f32) a valori interi a 8-bit (int8) tramite *Post-Training Quantization*.

Analisi dei Risultati e Footprint delle Risorse (Cortex-M4 @80MHz)

La piattaforma target utilizzata è il minuscolo modulo **SensorTile** (dotato di MCU STM32L476JG con core Cortex-M4F).

Grazie alla quantizzazione **int8**, l'occupazione di memoria si riduce di circa il 75% e la velocità di inferenza raddoppia, con una perdita di accuratezza inferiore all'1%. Il Duty Cycle complessivo stimato della MCU per l'intera pipeline è di appena il **13.6%**.

CASE STUDY 2: ACOUSTIC EVENT DETECTION (AED) + OOD SU STM32H7

L'**Acoustic Event Detection (AED)** si occupa di riconoscere specifici eventi sonori isolati (es. pianto di un bambino, allarme, vetri infranti) all'interno di un flusso audio continuo. A questo viene accoppiato un modulo **OOD (Out-of-Distribution)** per rilevare ed escludere i suoni sconosciuti non appartenenti alle classi di addestramento.

Apprendimento per Trasferimento (Transfer Learning) e Dataset

Invece di addestrare una rete da zero, si sfrutta il **Transfer Learning** partendo da modelli pre-addestrati su larga scala:

- **YAMNet:** Una rete profonda basata sull'architettura leggera *MobileNet_v1 depthwise-separable convolution*, addestrata sul corpus *AudioSet-YouTube* (comprendente più di 2 milioni di clip per 632 classi).
- **Dataset di Target (ESC-50):** Per l'applicazione specifica (es. Allarmi Domestici / *Home Alert Sounds*), il sistema è stato tarato e aumentato artificialmente (*Data Augmentation*) applicando tecniche di mix sintetico, stiramento temporale (*Time stretch*) e riverbero, portando il dataset a oltre 50.500 campioni da 1 secondo ciascuno.

Pipeline del Sistema AED+OOD Multi-Modello

Il sistema embedded adotta una strategia a modelli multipli per gestire classificazione e anomalia in parallelo:

Featurization: L'audio PCM (16kHz, mono) viene convertito in uno spettrogramma Mel 96x64 (finestre di 25ms, hop di 10ms, mappate su 64 bin Mel).

Feature Encoder (YAMNet tagliata a 256): Lo spettrogramma alimenta l'encoder YAMNet (quantizzato in **int8** per risparmiare risorse), che estrae un vettore di embedding compatto da 256 elementi (**256x1**).

Classificatore AED (Inference Head - f32): Un piccolo livello fully-connected riceve il vettore ed effettua la classificazione finale sulle classi note.

Rilevatore OOD (f32): Analizza l'embedding per determinare se il suono appartiene alla distribuzione nota (In-Distribution - IND) o se è un rumore esterno da rifiutare (OOD). L'introduzione di questa architettura ottimizzata ha migliorato l'accuratezza bilanciata del rilevamento delle anomalie del **+2.8%** rispetto alle baseline standard basate su soglie.

Profilazione Hardware del Caso d'Uso "Home Alert" (STM32H7 @480MHz)

Il sistema è profilato su 4 classi target (*Baby Crying, Door Knock, Glass Breaking, Clock Alarm*) con un periodo di esecuzione di 480ms:

- **YAMNet 256 (int8):** Richiede 136.62 KB di Flash, 124.88 KB di RAM e impiega **77.5 ms** per l'esecuzione (23.9M MACCs).
- **AED Classifier (f32):** Estremamente leggero. Richiede solo 4.02 KB di Flash, 1.03 KB di RAM e impiega appena **0.013 ms**.
- **OOD Detector (f32):** Richiede 5.04 KB di Flash, 1.03 KB di RAM e impiega **0.016 ms**.
- **Totale Risorse:** Flash complessiva: ~145.68 KB | RAM complessiva: ~126.94 KB | Tempo di calcolo totale: **~77.53 ms**.